Understanding Customer Conversion in Bank Marketing Campaigns

Project Report

This project was completed as part of the course:
DATA 606: Statistical Methods in Data Science.

Contributors:
Ishwarya Rani Murali (30271950)
Namya Dimri (30142290)
Sanjay Sundar (30299383)
Shreyaa Sathesh Kumar (30290646).

1. Overview

Banks invest heavily in direct marketing campaigns to promote financial products such as term deposits. However, subscription rates are typically low, making efficient customer targeting a critical challenge. Contacting customers with low likelihood of subscription increases operational costs and reduces campaign effectiveness.

With the availability of structured customer-level data, predictive analytics offers a data-driven approach to improve marketing strategies. By analyzing demographic characteristics, financial indicators, and prior campaign interactions, it becomes possible to identify patterns associated with successful subscriptions.

This project applies statistical analysis and machine learning techniques to examine customer conversion behavior. Logistic Regression and Random Forest models are implemented to evaluate how accurately subscription outcomes can be predicted and to identify the most influential factors driving customer decisions.

2. Dataset

The analysis uses the **Bank Marketing dataset** from the UCI Machine Learning Repository.

The dataset contains:

- **41,188 customer records**
- **21 predictor variables**
- A binary target variable **(y)** indicating whether a customer subscribed to a term deposit

The data were collected from telemarketing campaigns conducted between **2008 and 2010** by a Portuguese banking institution. Each record represents one customer contact during a campaign.

Variable Categories

The dataset includes:

- **Demographic variables** (age, job, marital status, education)
- **Financial indicators** (housing loan, personal loan, default status, macroeconomic variables)
- **Campaign-related features** (number of contacts, contact method, previous campaign outcomes, month, and day of contact)

3. Objective

The main objective of this project is to analyze the factors influencing customer subscription to term deposits and to evaluate how effectively subscription outcomes can be predicted.

Specifically, the study aims to:

- Examine how demographic, financial, and campaign-related variables affect subscription decisions
- Identify the most important predictors of term deposit subscription
- Develop and compare classification models (Logistic Regression and Random Forest)
- Evaluate model performance using Accuracy, Precision, Recall, and F1-score

The central question guiding this analysis is:

Which factors most strongly influence customer subscription, and how reliably can predictive models identify potential subscribers in an imbalanced real-world dataset?

4. Preprocessing:

The dataset contains both categorical and numerical features. For clarity:

- Categorical Features
  - job, marital, education, default, housing, loan, contact, month, day_of_week, poutcome
  - These features represent discrete categories such as job type, marital status, or previous marketing outcomes.

- Numerical Features
  - age, campaign, pdays, previous, emp.var.rate, cons.price.idx, cons.conf.idx, euribor3m, nr.employed
  - These features are continuous variables representing numerical measurements, including demographic data, economic indicators, and marketing campaign statistics.

- Target Variable: y, indicating the outcome of the marketing campaign (e.g., whether the client subscribed to a term deposit).

Preprocessing Steps:

- Categorical Data Processing: Categorical features were transformed using one-hot encoding, converting each category into a binary vector.

- Numerical Data Processing: Numerical features were standardized, i.e., rescaled to have a mean of zero and a standard deviation of one.

- Target Variable Transformation: The target variable y was converted to a binary format, where positive and negative campaign outcomes are represented as 1 and 0, respectively.

- The *duration* feature, which measures the length of the last contact with a client, was removed. Since this information is only available after the marketing campaign is conducted, including it during model training would introduce data leakage.

1. EDA:

The graph indicates a pronounced class imbalance in the dataset. Specifically, the number of customers who subscribed to a term deposit is substantially lower than those who did not, highlighting that successful outcomes are relatively rare. This imbalance must be carefully addressed during model development to ensure that predictive performance is not biased toward the majority class.

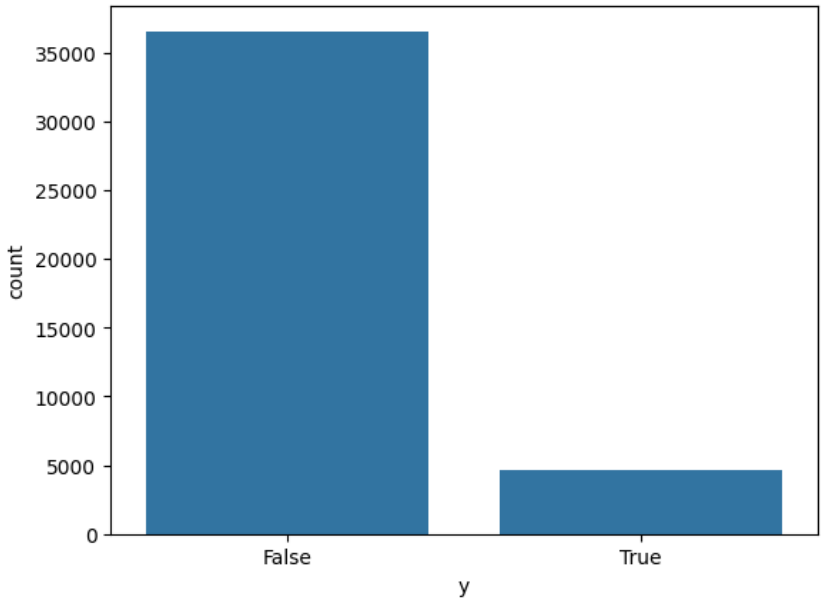

Fig. Class Imbalance

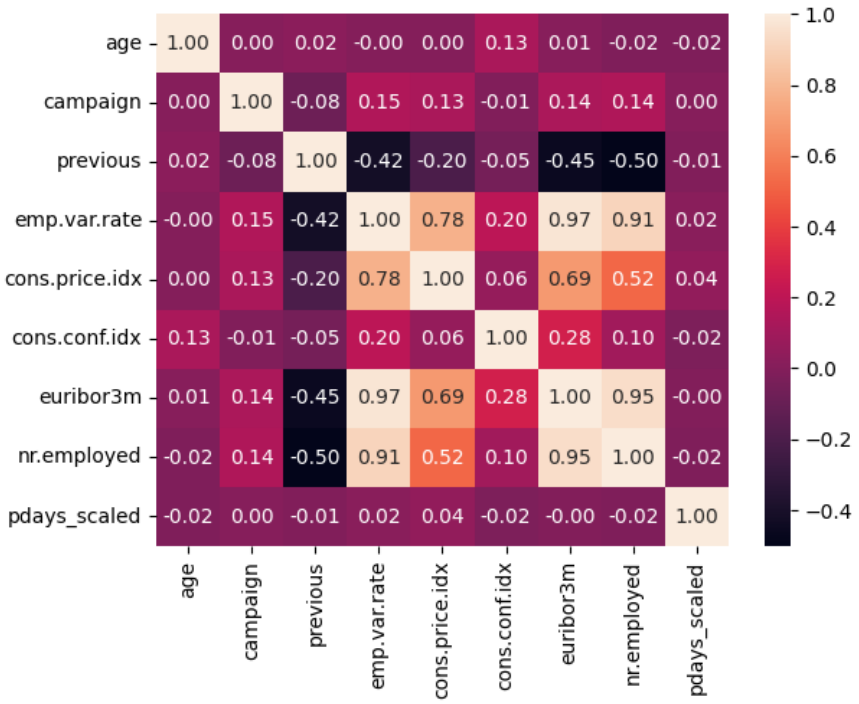

Fig. Correlation Heatmap

The correlation matrix shows that several social and economic features are strongly related to each other. Notably:

- Strong positive correlations exist among economic indicators: emp.var.rate and euribor3m (0.97), nr.employed and euribor3m (0.95), emp.var.rate and nr.employed (0.91), and emp.var.rate and cons.price.idx (0.78).

- Moderate negative correlations are observed between previous interactions and economic variables: previous and emp.var.rate (-0.42), euribor3m (-0.45), and nr.employed (-0.50).

These strong relationships indicate that some features carry overlapping information, which could influence model performance and should be considered during feature selection or regularization.

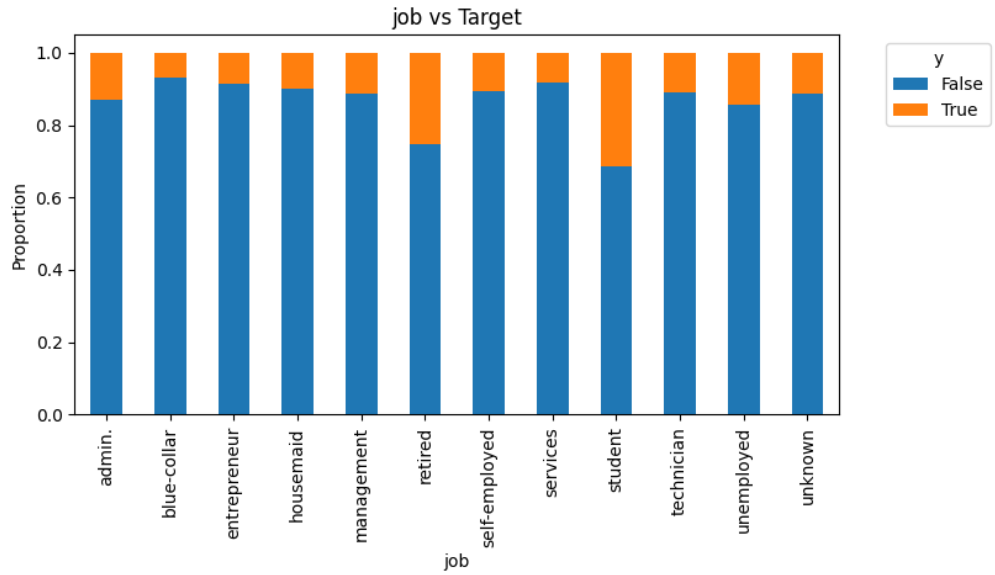

Fig. Job category vs Target y

From the graph, students and retired people have the highest conversion ratios.

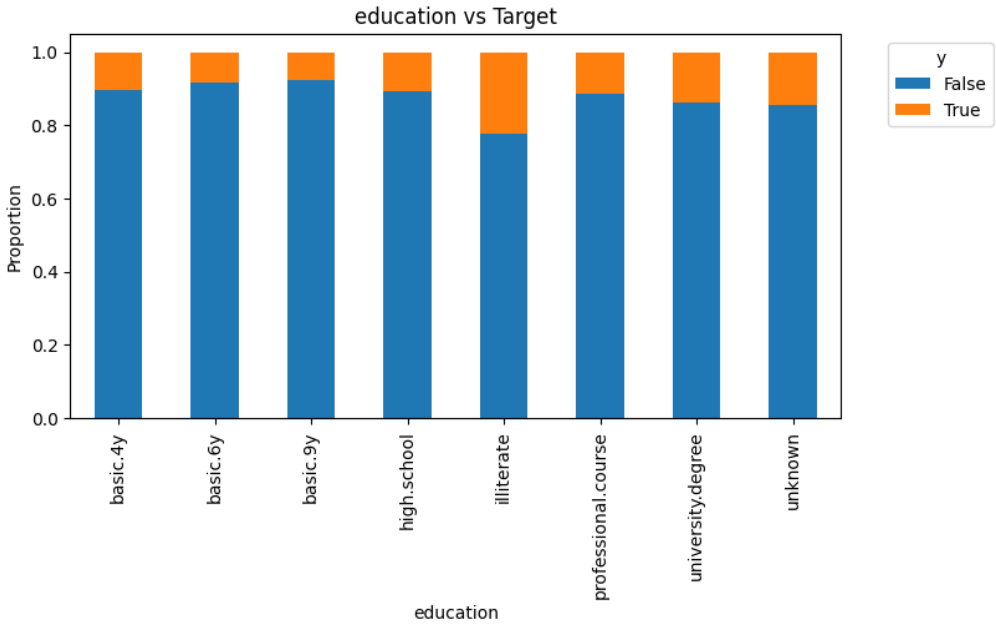

Fig. Education levels vs Target y

Customers without formal education show a higher likelihood of subscribing.

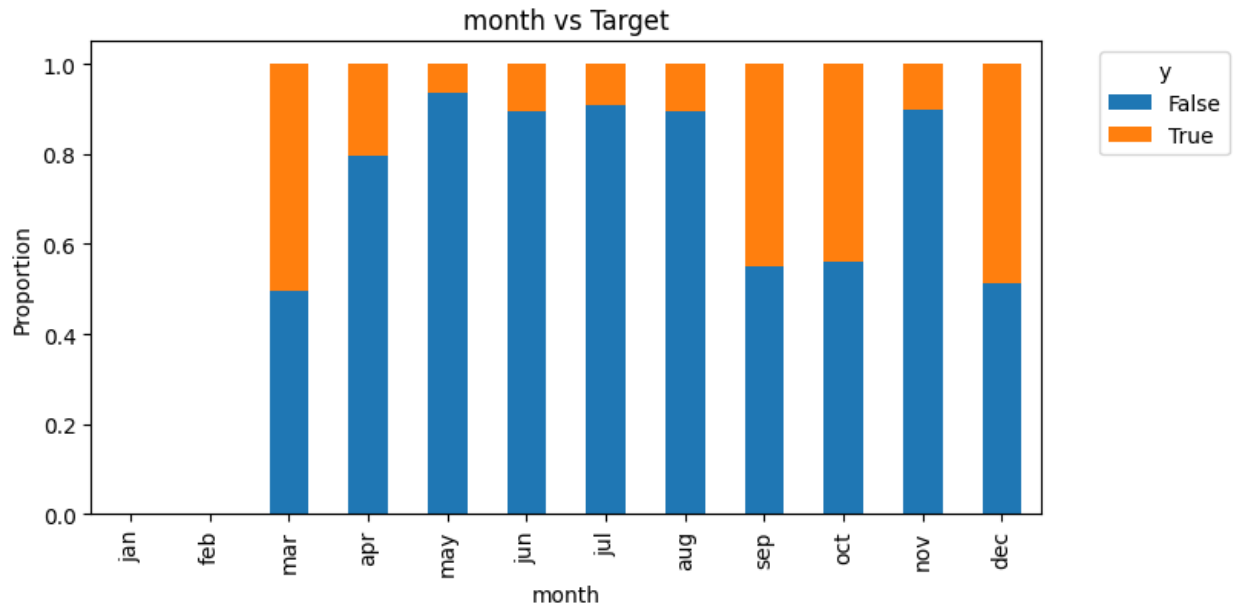

Fig. Months vs Target y

Most customers subscribed to a term deposit in the months of March, September, October and December. This pattern may reflect seasonal financial behaviors, such as end-of-quarter planning, post-summer savings adjustments, or year-end bonuses.

2. Predicting Customer Subscription Outcomes:

The primary goal of this analysis was to evaluate how accurately we can predict whether a bank customer will subscribe to a service. Two machine learning models: Logistic Regression and Random Forest Classification, were applied and optimized to assess predictive performance. The analysis focused on both overall accuracy and the ability to correctly identify the minority class (customers who subscribe).

Models Evaluated:

- Logistic Regression: Evaluated for baseline performance and feature interpretability.
- Random Forest Classification: Evaluated for higher accuracy and robustness to non-linear patterns.

Hyperparameter Tuning:

GridSearchCV was used for both models to optimize hyperparameters and improve predictive performance.

- Logistic Regression Parameters Searched:
    - C (inverse regularization strength): [0.001, 0.01, 0.1, 1, 10, 100]
    - penalty (type of regularization): ['l1', 'l2']
    - Best parameters: {'C': 0.001, 'penalty': 'l1'}
- Random Forest Parameters Searched:
    - n_estimators (number of trees): [100, 200, 300]
    - max_depth (maximum depth of the tree): [None, 5, 10, 20]
    - min_samples_split (minimum samples required to split a node): [2, 5, 10]
    - max_features (number of features to consider for best split): ['sqrt', 'log2', None]
    - Best parameters: {'max_depth': None, 'max_features': 'log2', 'min_samples_split': 2, 'n_estimators': 300}
    - max_depth, max_features, min_samples_split, n_estimators
    - Best parameters: {'max_depth': None, 'max_features': 'log2', 'min_samples_split': 2, 'n_estimators': 300}

Class imbalance was addressed using class_weight='balanced' during model training.

Model Performance:

- Logistic Regression:

Class	Precision	Recall	F1-score
False (No Subscription)	0.91	0.89	0.90
True (Subscription)	0.26	0.31	0.29

Table: Performances of Logistic Regression

  - Cross-Validation Accuracy: 0.8232
  - Test Set Accuracy: 0.8248

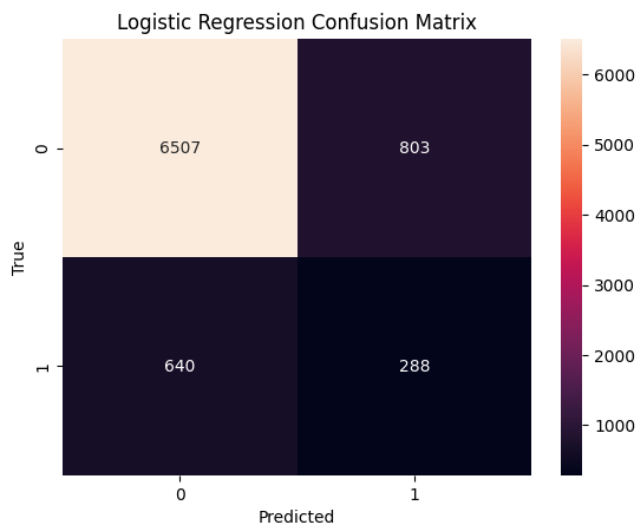

Fig. Confusion Matrix of Logistic Regression

  - High overall accuracy is largely due to the dominant 'False' class.
  - Poor performance in predicting the 'True' class (subscribers), indicated by low precision (0.26) and recall (0.31).
  - False Positives (803) and False Negatives (640) highlight challenges in minority class prediction.

- Random Forest

Class	Precision	Recall	F1-score
False (No Subscription)	0.91	0.97	0.94
True (Subscription)	0.54	0.26	0.35

Table: Performances of Random Forest Classification

- Cross-Validation Accuracy: 0.8894
- Test Set Accuracy: 0.8918

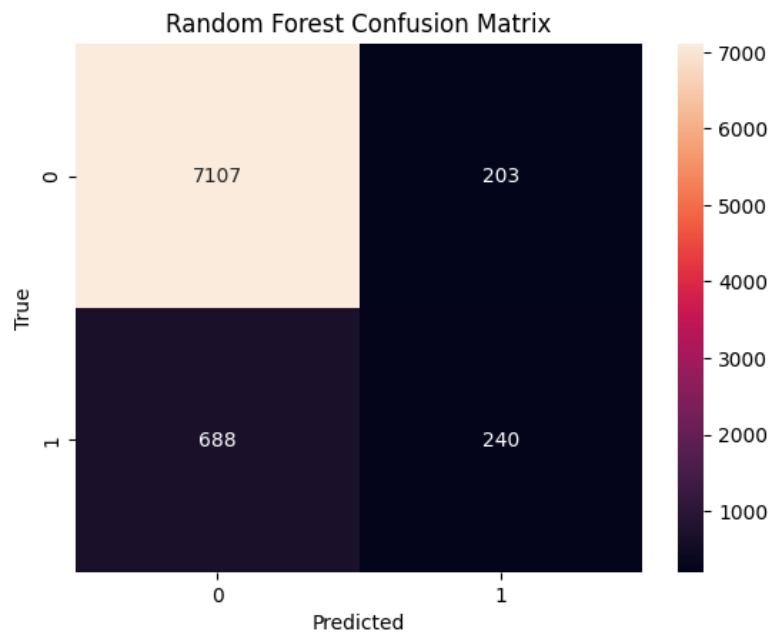

Fig. Confusion Matrix of Random Forest Classification

- Higher overall accuracy compared to Logistic Regression.
- Precision for the 'True' class improved significantly (0.54 vs. 0.26), reducing unnecessary targeting of non-subscribers.

- Recall for the 'True' class slightly lower (0.26 vs. 0.31), meaning some potential subscribers are missed.
- False Positives reduced to 203, while False Negatives slightly increased to 688.

Model Comparison:

Metric	Logistic Regression	Random Forest
Overall Accuracy	0.8248	0.8918
Precision (True Class)	0.26	0.54
Recall (True Class)	0.31	0.26
F1-score (True Class)	0.29	0.35
False Positives (FP)	803	203
False Negatives (FN)	640	688

Table: Performance comparison

Overall Accuracy: Random Forest outperforms Logistic Regression, but high accuracy can be misleading due to dataset imbalance.

Precision vs. Recall Trade-off: Random Forest prioritizes precision (fewer false alarms), suitable when cost of contacting non-subscribers is high. Logistic Regression has slightly better recall, identifying more actual subscribers but with more false positives.

F1-score Improvement: Random Forest shows better balance between precision and recall for the minority class.

Insights from Customer Subscription Prediction

The analysis demonstrates that predicting customer subscription outcomes remains challenging due to the highly imbalanced nature of the dataset, where non-subscribers significantly outnumber subscribers. While

hyperparameter tuning and class balancing improved model performance, identifying potential subscribers with high confidence continues to be difficult. The Random Forest model outperformed Logistic Regression in terms of overall accuracy (0.8918 vs. 0.8248) and precision for the subscriber class (0.54 vs. 0.26), indicating a more reliable identification of likely subscribers and a reduction in wasted marketing efforts. However, recall for subscribers remained relatively low (0.26 for Random Forest, 0.31 for Logistic Regression), suggesting that a notable portion of potential subscribers are still being missed.

These results highlight a clear trade-off between precision and recall: Random Forest prioritizes precision, minimizing false positives and avoiding unnecessary outreach, whereas Logistic Regression captures slightly more subscribers at the cost of a higher number of false positives. To further enhance predictive performance, especially for the minority class, additional strategies could be employed, such as oversampling techniques like SMOTE, advanced ensemble methods, targeted feature engineering, and adjustment of decision thresholds.

In conclusion, while both models provide valuable insights, Random Forest is the preferred model for practical deployment, given its superior overall accuracy and precision. Nonetheless, further optimization is recommended to reliably identify potential subscribers while minimizing missed opportunities, ensuring more effective and cost-efficient customer targeting.

3. Feature Importance Analysis

Feature importance analysis was conducted using the Random Forest model to identify the most influential predictors of term deposit subscription.

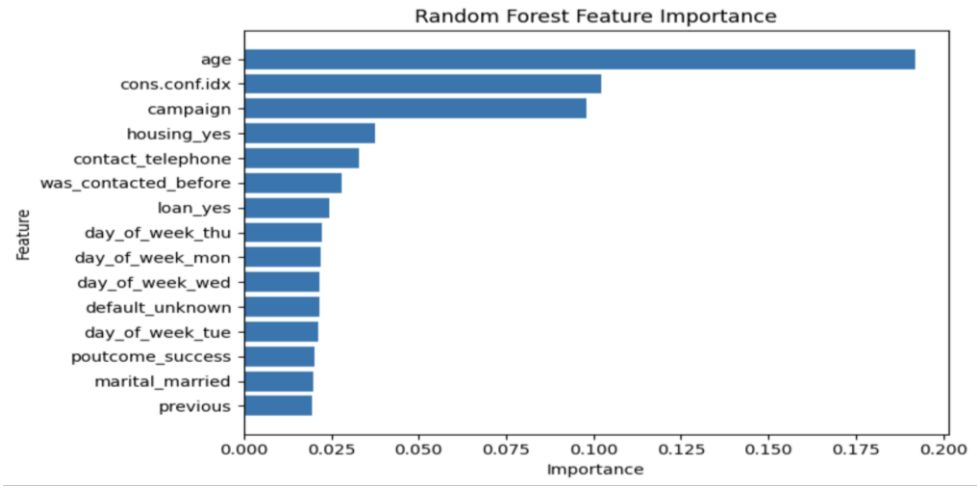

Fig. Random forest feature importance

	Feature	Importance
0	age	0.191866
3	cons.conf.idx	0.102337
1	campaign	0.097803
28	housing_yes	0.037667
31	contact_telephone	0.032798
47	was_contacted_before	0.027825
30	loan_yes	0.024369
42	day_of_week_thu	0.022334
41	day_of_week_mon	0.021843
44	day_of_week_wed	0.021601
25	default_unknown	0.021499
43	day_of_week_tue	0.021211
46	poutcome_success	0.020342
15	marital_married	0.020005
2	previous	0.019427

Table: Top 15 Important Features

The results indicate that age is the most dominant predictor, followed by consumer confidence index, campaign intensity (number of contacts), previous campaign outcome, and broader employment-related macroeconomic indicators.

The prominence of age suggests that demographic segmentation plays a critical role in determining subscription likelihood, potentially reflecting differences in financial goals, income stability, and risk tolerance across life stages. The strong contribution of the consumer confidence index highlights the influence of macroeconomic sentiment on individual financial decision-making, implying that customers may be more inclined to commit to long-term deposits when economic outlook and consumer optimism are favorable.

Additionally, the importance of the campaign variable indicates that the number of contacts significantly affects subscription probability; however, the relationship likely reflects diminishing marginal returns, where excessive contact may reduce effectiveness rather than enhance it. The substantial influence of previous campaign outcomes further underscores the predictive power of prior engagement history, as customers who previously responded positively are considerably more likely to subscribe again.

Finally, employment-related indicators reinforce the role of macroeconomic stability in shaping financial behavior. Collectively, these findings are consistent with behavioral economics principles, suggesting that financial decisions are shaped not only by individual demographic characteristics but also by broader economic conditions and prior interaction experiences.

4. Conclusion

In addressing the central research question: which factors most strongly influence customer subscription, and how reliably can predictive models identify potential subscribers in an imbalanced real-world dataset, this study provides both substantive and methodological insights.

The analysis demonstrates that customer subscription behavior is most strongly influenced by a combination of demographic characteristics particularly age, prior campaign interaction history, campaign intensity, and macroeconomic sentiment indicators such as consumer confidence. These factors collectively reveal that financial decision-making is not driven by isolated attributes but rather by the interaction between individual profile characteristics and broader economic conditions. In particular, previous successful contact and economic confidence emerge as powerful signals of readiness to commit to a term deposit.

From a modeling perspective, the study confirms that predictive models can reliably identify potential subscribers despite significant class imbalance, provided that appropriate preprocessing, multicollinearity handling, and hyperparameter tuning strategies are employed. While Logistic Regression offered interpretability and structured statistical inference, its linear framework limited its ability to capture complex behavioral patterns.

In contrast, the Random Forest model achieved superior predictive performance and improved minority class precision, indicating the presence of meaningful non-linear relationships within the dataset. The model's ability to generalize well to unseen data further supports its robustness in a real-world marketing context.

Strategically, these findings carry important implications for campaign design. Marketing efforts should prioritize age segments with higher predicted conversion probabilities, optimize contact frequency to avoid diminishing marginal returns, and incorporate macroeconomic indicators into campaign timing decisions. Additionally, prior successful engagement should be leveraged as a key retention and cross-selling strategy.

Overall, this study highlights the transformative value of data-driven marketing within financial services. By integrating advanced machine learning models into customer targeting and resource allocation decisions, financial institutions can move beyond intuition-based outreach toward analytically grounded strategies that enhance conversion efficiency, reduce

operational costs, and build more sustainable customer relationships in competitive and imbalanced market environments.

5. Limitations

Despite the strong predictive performance achieved in this study, several limitations must be acknowledged to provide a balanced and academically rigorous evaluation of the findings. First, the dataset exhibits a pronounced class imbalance, with non-subscribers substantially outnumbering subscribers. Although techniques such as class weighting were implemented to mitigate this issue, the imbalance inherently constrains the model's ability to accurately identify the minority class, resulting in relatively modest recall for subscribers. From a business perspective, this limitation is significant because missed positive cases represent lost revenue opportunities.

Second, the macroeconomic variables included in the dataset such as consumer confidence and employment indicators are time-sensitive and context-dependent. Their predictive relevance may vary across economic cycles, geographic regions, and financial environments. Consequently, models trained on data from a specific temporal and national context may not generalize effectively to different macroeconomic conditions or markets without recalibration. This temporal and spatial variability restricts the stability of model performance over time.

Third, while Random Forest feature importance scores provide valuable insights into variable contribution, they do not establish causal relationships. High importance indicates strong predictive association but does not confirm that a given variable directly causes subscription behavior. As such, interpretations should be made cautiously and ideally supplemented with econometric or experimental methods to validate causal inferences.

Additionally, the dataset originates from a single Portuguese banking institution, raising concerns regarding external validity. Customer demographics, regulatory frameworks, marketing practices, and economic conditions differ across institutions and countries, which may limit the generalizability of the findings to broader financial contexts. Finally, although

Random Forest improved precision for the subscriber class, recall remains comparatively low, indicating that a substantial proportion of potential subscribers are still misclassified.

6. Future works

Future research should focus on methodological enhancements aimed at improving minority class detection, model robustness, and interpretability.

Given the pronounced class imbalance observed in the dataset, the application of advanced resampling techniques such as Synthetic Minority Over-sampling Technique (SMOTE) or its variants could help generate synthetic examples of the minority class, thereby improving recall for subscribers without excessively compromising precision.

Beyond resampling, experimentation with more advanced ensemble methods particularly Gradient Boosting algorithms such as XGBoost or LightGBM may yield superior predictive performance by capturing complex non-linear interactions and optimizing error reduction sequentially. These boosting frameworks often outperform traditional bagging approaches in structured tabular datasets and may enhance both discrimination and calibration of predicted probabilities.

Additionally, future work should consider optimizing the classification threshold in alignment with business objectives. Rather than relying on the default probability cutoff of 0.5, threshold tuning could prioritize recall for subscribers if minimizing missed conversions is strategically more important than reducing false positives. Such optimization would align model deployment more closely with real-world marketing cost-benefit trade-offs. To further enhance interpretability and transparency, SHAP (SHapley Additive exPlanations) analysis could be conducted to quantify the marginal contribution of each feature at both global and individual prediction levels. This would provide deeper insight into how specific variables influence model decisions and improve stakeholder trust.

Finally, implementing cost-sensitive learning frameworks that explicitly incorporate misclassification costs into the training process could allow the

model to balance precision and recall according to financial impact, thereby making predictive outcomes more economically aligned with institutional objectives. Collectively, these extensions would strengthen the model's practical applicability, interpretability, and strategic value in banking marketing analytics

7. References

1. *Moro, S., Cortez, P., & Rita, P. (2014). A data-driven approach to predict the success of bank telemarketing. Decision Support Systems, 62, 22–31 - https://www.sciencedirect.com/science/article/abs/pii/S016792361400061X*

2. *Safarkhani, F., & Moro, S. (2021). Improving the accuracy of predicting bank depositor's behavior using a decision tree. Applied Sciences, 11(19), 9016 - https://www.mdpi.com/2076-3417/11/19/9016*

3. *Zhao, Z., et al. (2025). Machine learning approaches for predicting bank telemarketing subscription. ITM Conferences (classification study including logistic regression and random forest) - https://www.itm-conferences.org/articles/itmconf/pdf/2025/04/itmconf_iwadi2024_02020.pdf*

4. *UCI Machine Learning Repository - Bank Marketing Dataset (Dataset source) - https://archive.ics.uci.edu/ml/datasets/bank+marketing*

APPENDIX

Loading the dataset:

```
df= pd.read_csv('bank-additional-full.csv',sep=';')
df.head()
```

Dropping Column:

```
#dropping column duration
df = df.drop('duration', axis=1)
```

Converting Target Variable to Binary:

```
df['y']= df['y']=="yes"
df_orig= df.copy()
df.head()
```

One Hot Encoding:

```
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
# Exclude 'y' if it's still in object dtype, though it should be int/bool now
if 'y' in categorical_cols:
    categorical_cols.remove('y')

df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

df = df_encoded
df.head()
```

Scaling Numeric Features:

```
numeric_cols = ['age', 'campaign', 'previous',
                'emp.var.rate', 'cons.price.idx', 'cons.conf.idx',
                'euribor3m', 'nr.employed', 'pdays_scaled']

scaler = StandardScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
```

Proportion of Successful Outcomes by Category:

```
cat_cols = ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'day_of_week', 'poutcome']

for col in cat_cols:
    counts = df_orig.groupby([col, 'y']).size().unstack(fill_value=0)
    proportions = counts.div(counts.sum(axis=1), axis=0)  # normalize per category

    proportions.plot(kind='bar', stacked=True, figsize=(8,4))
    plt.title(f'{col} vs Target')
    plt.ylabel('Proportion')
    plt.legend(title='y', bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.show()
```

Removing Highly collinear variables:

```
# removing higly collinear variables:
Features  = df_modelling.drop('y', axis=1)
X = Features.drop(columns=[
    'nr.employed',
    'euribor3m',
    'emp.var.rate',
    'cons.price.idx'
])
y = df_modelling['y']
```

Splitting data for training and validation:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Grid search for Logistic regression:

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression

# 1. Define the parameter grid
param_grid_log_reg = {
    'C': [0.001, 0.01, 0.1, 1, 10, 100],
    'penalty': ['l1', 'l2']
}

# 2. Initialize a LogisticRegression model
log_reg = LogisticRegression(solver='liblinear', random_state=42, max_iter=1000, class_weight='balanced')

# 3. Initialize a GridSearchCV object
grid_search_log_reg = GridSearchCV(estimator=log_reg, param_grid=param_grid_log_reg, cv=5, scoring='accuracy', n_jobs=-1)
```

```
grid_search_log_reg.fit(X_train, y_train)
```

```
GridSearchCV
best_estimator_:
LogisticRegression
LogisticRegression
```

Best parameters for logistic regression:

```
print("Best parameters found: ", grid_search_log_reg.best_params_)
print("Best accuracy found: ", grid_search_log_reg.best_score_)
```

```
Best parameters found:  {'C': 0.001, 'penalty': 'l1'}
Best accuracy found:  0.8232473444613049
```

Predicting on validation set:

```
best_log_reg = grid_search_log_reg.best_estimator_
y_pred = best_log_reg.predict(X_test)
```

Performance on Validation set:

```python
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}")
print("Classification Report:")
print(report)
print("Confusion Matrix:")
print(conf_matrix)
```

```
Accuracy: 0.8248
Classification Report:
              precision    recall  f1-score   support

       False       0.91      0.89      0.90      7310
        True       0.26      0.31      0.29       928

    accuracy                           0.82      8238
   macro avg       0.59      0.60      0.59      8238
weighted avg       0.84      0.82      0.83      8238

Confusion Matrix:
[[6507  803]
 [ 640  288]]
```

Defining Grid for Random Forest classification:

```python
from sklearn.ensemble import RandomForestClassifier

# Define the parameter grid for Random Forest
param_grid_rf = {
    'n_estimators': [100, 200, 300],
    'max_features': ['sqrt', 'log2'],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5, 10]
}
```

Grid search for Random Forest Classification:

```python
from sklearn.model_selection import GridSearchCV

# Initialize a RandomForestClassifier model with class_weight='balanced'
rf_classifier = RandomForestClassifier(class_weight='balanced', random_state=42)

# Initialize a GridSearchCV object for Random Forest
grid_search_rf = GridSearchCV(estimator=rf_classifier, param_grid=param_grid_rf, cv=5, scoring='accuracy', n_jobs=-1)
```

```python
grid_search_rf.fit(X_train, y_train)
```

```
GridSearchCV for Random Forest fitted successfully.
```

Best parameter for Random Forest Classification:

```python
print("Best parameters found: ", grid_search_rf.best_params_)
print("Best accuracy found: ", grid_search_rf.best_score_)
```

```
Best parameters found:  {'max_depth': None, 'max_features': 'log2', 'min_samples_split': 2, 'n_estimators': 300}
Best accuracy found:  0.8893778452200303
```

Predictions on the validation set:

```python
best_rf_classifier = grid_search_rf.best_estimator_
y_pred_rf = best_rf_classifier.predict(X_test)
print("Predictions on X_test made successfully.")
```

```
Predictions on X_test made successfully.
```

Performance on the validation set:

```python
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Evaluate the Random Forest model
accuracy_rf = accuracy_score(y_test, y_pred_rf)
report_rf = classification_report(y_test, y_pred_rf)
conf_matrix_rf = confusion_matrix(y_test, y_pred_rf)

print(f"Random Forest Accuracy: {accuracy_rf:.4f}")
print("Random Forest Classification Report:")
print(report_rf)
print("Random Forest Confusion Matrix:")
print(conf_matrix_rf)
```

```
Random Forest Accuracy: 0.8918
Random Forest Classification Report:
              precision    recall  f1-score   support

       False       0.91      0.97      0.94      7310
        True       0.54      0.26      0.35       928

    accuracy                           0.89      8238
   macro avg       0.73      0.62      0.65      8238
weighted avg       0.87      0.89      0.87      8238

Random Forest Confusion Matrix:
[[7107  203]
 [ 688  240]]
```

Feature Importance

```python
# Get feature importance
importances = best_rf_classifier.feature_importances_

# Create dataframe
feature_importance = pd.DataFrame({
    "Feature": X_train.columns,
    "Importance": importances
})

# Sort values
feature_importance = feature_importance.sort_values(
    by="Importance", ascending=False
)

# Show top 15 features
print("Top 15 Important Features:")
display(feature_importance.head(15))

# Plot
plt.figure(figsize=(8,5))
plt.barh(
    feature_importance["Feature"].head(15)[::-1],
    feature_importance["Importance"].head(15)[::-1]
)
plt.title("Random Forest Feature Importance")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```